

Aquí tienes el documento estructurado. Puedes copiarlo y guardarlo como un archivo de texto o un documento técnico de referencia para tu equipo.

Plan de Actualización Estructural para Tablas Masivas en SQL Server

Objetivo: Modificar tipos de datos en tablas con millones de filas minimizando el crecimiento del Transaction Log y evitando bloqueos prolongados.

1. Diagnóstico y Preparación

Antes de tocar la tabla, es vital entender el entorno para evitar el error **3023** (operaciones serializadas).

- **Verificar procesos activos:** Asegurarse de que no haya backups o tareas de integridad (DBCC CHECKDB) corriendo.

SQL

```
SELECT session_id, command, percent_complete FROM sys.dm_exec_requests  
WHERE command LIKE '%BACKUP%' OR command LIKE '%DBCC%';
```

- **Revisar el Log:** El log debe tener espacio para crecer y estar en un disco con capacidad. Se recomienda crecimiento fijo (ej. 512MB) y no por porcentaje.
-

2. Fase de Estructura (Shadow Column)

En lugar de modificar la columna existente (que bloquea la tabla y reescribe todo el log de una vez), creamos una columna auxiliar.

- **Acción:** Agregar la nueva columna permitiendo valores NULL.
- **Ejemplo:**

SQL

```
-- Caso: Ampliar precisión de precio por inflación  
ALTER TABLE cuerfac ADD precio_nuevo NUMERIC(13,3) NULL;
```

3. Fase de Migración (Data Batching)

Esta es la parte crítica para mantener el servidor estable. Dividimos la carga en "lotes".

- **Acción:** Ejecutar un bucle que actualice de a N filas (ej. 50,000).
- **Script de Ejemplo:**

```
SQL
SET NOCOUNT ON;
DECLARE @RowsAffected INT = 1;
DECLARE @Batch INT = 0;

WHILE @RowsAffected > 0
BEGIN
    UPDATE TOP (50000) cuerfac
    SET precio_nuevo = precio -- Migración del dato
    WHERE precio_nuevo IS NULL; -- Filtro para avanzar

    SET @RowsAffected = @@ROWCOUNT;
    SET @Batch = @Batch + 1;

    PRINT 'Lote ' + CAST(@Batch AS VARCHAR) + ' procesado.';

    -- Si el log se llena, aquí se puede insertar un BACKUP LOG (si está en modo FULL)
END
```

4. Fase de Dependencias e Intercambio (Swapping)

Una vez que la columna nueva está llena, debemos reemplazar la vieja.

- **Paso A: Scriptar Índices.** Buscar qué índices usan la columna vieja.

```
SQL
EXEC sp_helpindex 'cuerfac';
```

- **Paso B: Borrar Dependencias.** Borrar índices y constraints asociados a la columna precio.
- **Paso C: El Intercambio.**

```
SQL
-- 1. Eliminar la columna vieja (ya no tiene carga de datos masiva)
ALTER TABLE cuerfac DROP COLUMN precio;

-- 2. Renombrar la nueva para que el sistema no note el cambio
EXEC sp_rename 'cuerfac.precio_nuevo', 'precio', 'COLUMN';
```

5. Fase de Estabilización

- **Recrear Índices:** Volver a crear los índices que borramos en el Paso 4.B (ahora serán más

eficientes y nacerán sin fragmentación).

- **Actualizar Estadísticas:** Crucial para que las consultas sigan siendo rápidas.

SQL

```
UPDATE STATISTICS cuerfac;
```

Consideraciones Importantes (Lecciones Aprendidas)

1. **Modelo de Recuperación:** Si es posible, realizar esto con la DB en modo SIMPLE. Si es FULL, los backups de log deben ser muy frecuentes durante la Fase 3.
2. **Espacio en Disco:** La tabla ocupará un ~30% más de espacio temporalmente durante la migración.
3. **Rollback:** Si el proceso de batches se cancela, lo que ya se actualizó **queda** actualizado. Solo debes reiniciar el script y continuará donde dejó (gracias al filtro IS NULL).

Documento generado para referencia técnica - SQL Server DBA Checklist.